

Industrial Internship Report on “Foodie – A Full-Stack Online Food Delivery Web Application”

Prepared by

Shantanu

B.Tech, 3rd Year

National Institute of Technology Delhi (NIT Delhi)

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks’ time.

My project was to design and develop “**Foodie**”, a full-stack online food delivery web application that connects customers, restaurants and delivery partners on a single platform. The system includes a customer-facing web app for browsing restaurants and menus, a secure authentication and cart module, an order and payment pipeline integrated with Razorpay, and real-time order/delivery tracking built with Socket.io. The backend was built using Node.js, Express and MongoDB, with Cloudinary used for image storage.

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

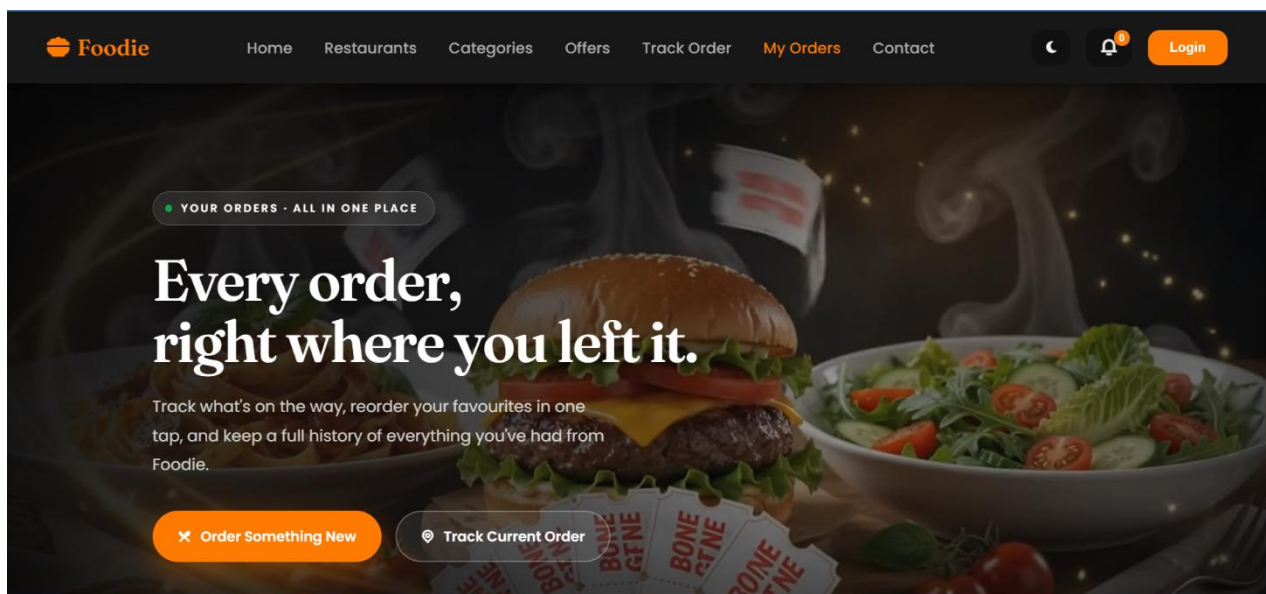
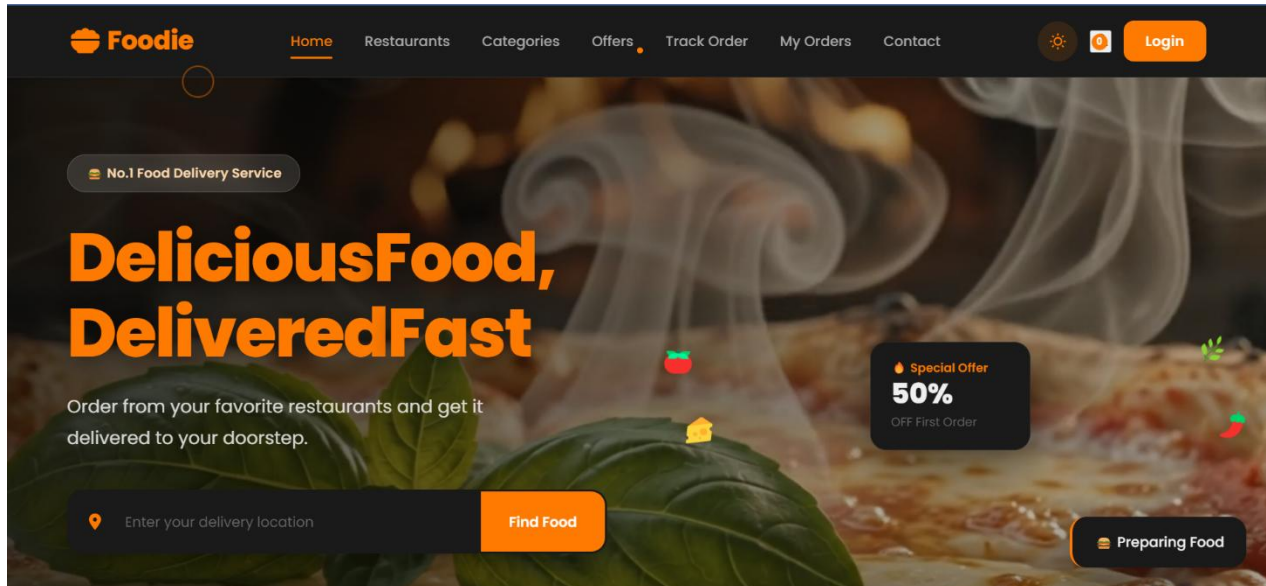


TABLE OF CONTENTS

1	Preface	4
2	Introduction	6
2.1	About UniConverge Technologies Pvt Ltd	6
2.2	About upskill Campus	10
2.3	Objective	12
2.4	Reference	12
2.5	Glossary.....	12
3	Problem Statement	14
4	Existing and Proposed solution.....	15
5	Proposed Design/ Model	16
5.1	High Level Diagram (if applicable)	16
5.2	Low Level Diagram (if applicable)	16
5.3	Interfaces (if applicable)	17
6	Performance Test.....	18
6.1	Test Plan/ Test Cases	18
6.2	Test Procedure	18
6.3	Performance Outcome	19
7	My learnings.....	20
8	Future work scope	21

1 Preface

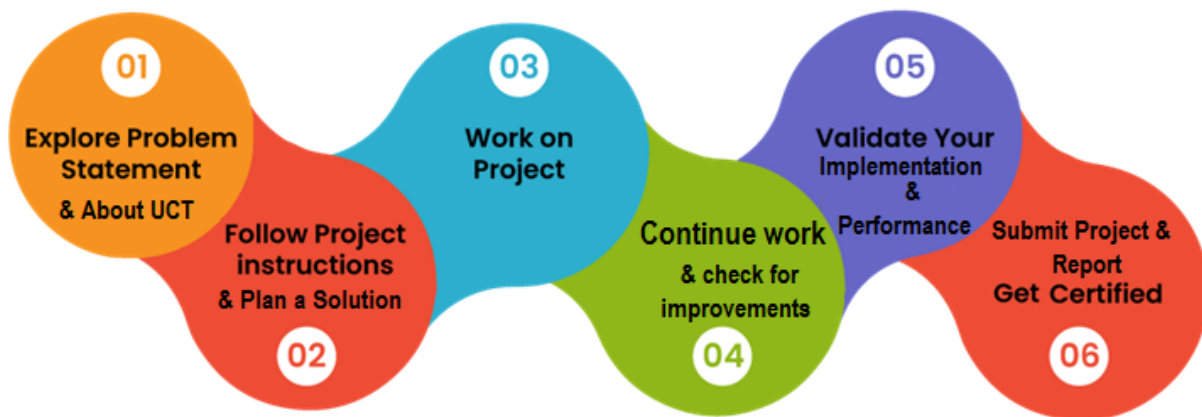
This report summarizes the 6-week Industrial Internship undertaken with upskill Campus, The IoT Academy and UniConverge Technologies Pvt Ltd (UCT), during which I designed and built a full-stack online food delivery web application.

An industrial internship like this is an essential bridge between academic learning and real-world engineering practice. It exposes a student to production-grade coding standards, end-to-end system design, and the discipline of delivering a working product within a fixed timeline — skills that a classroom setting alone cannot fully provide.

My project, “Foodie,” is an online food ordering and delivery platform where customers can browse restaurants and menus, add items to a cart, pay online, and track their order in real time, while restaurants and delivery partners get the tools they need to fulfil those orders efficiently.

USC and UCT gave me this opportunity to work on an industry-defined problem statement instead of a purely academic one, along with mentorship and structured milestones to guide the execution.

The program was planned across six stages as shown below — starting with understanding the problem statement, moving through design, implementation, iteration, validation, and finally submission of the project and report for certification.



Over these six weeks I learned how to architect a multi-module backend with Node.js, Express and MongoDB, secure it with JWT-based authentication, integrate a real payment gateway (Razorpay), and add real-time features with Socket.io. I also improved at debugging, writing modular code, and managing a project end-to-end instead of just a single assignment — an experience that has significantly boosted my confidence as a developer.

I would like to thank the mentors at upskill Campus, The IoT Academy and UniConverge Technologies for their guidance throughout this internship, and my faculty at NIT Delhi for their continued support and encouragement.

To my juniors and peers: take up this internship seriously — it is one of the few opportunities where you build something real, get direct feedback from industry mentors, and walk away with a project you can proudly showcase. Start early, ask questions, and don't be afraid to break and rebuild your code until it works.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



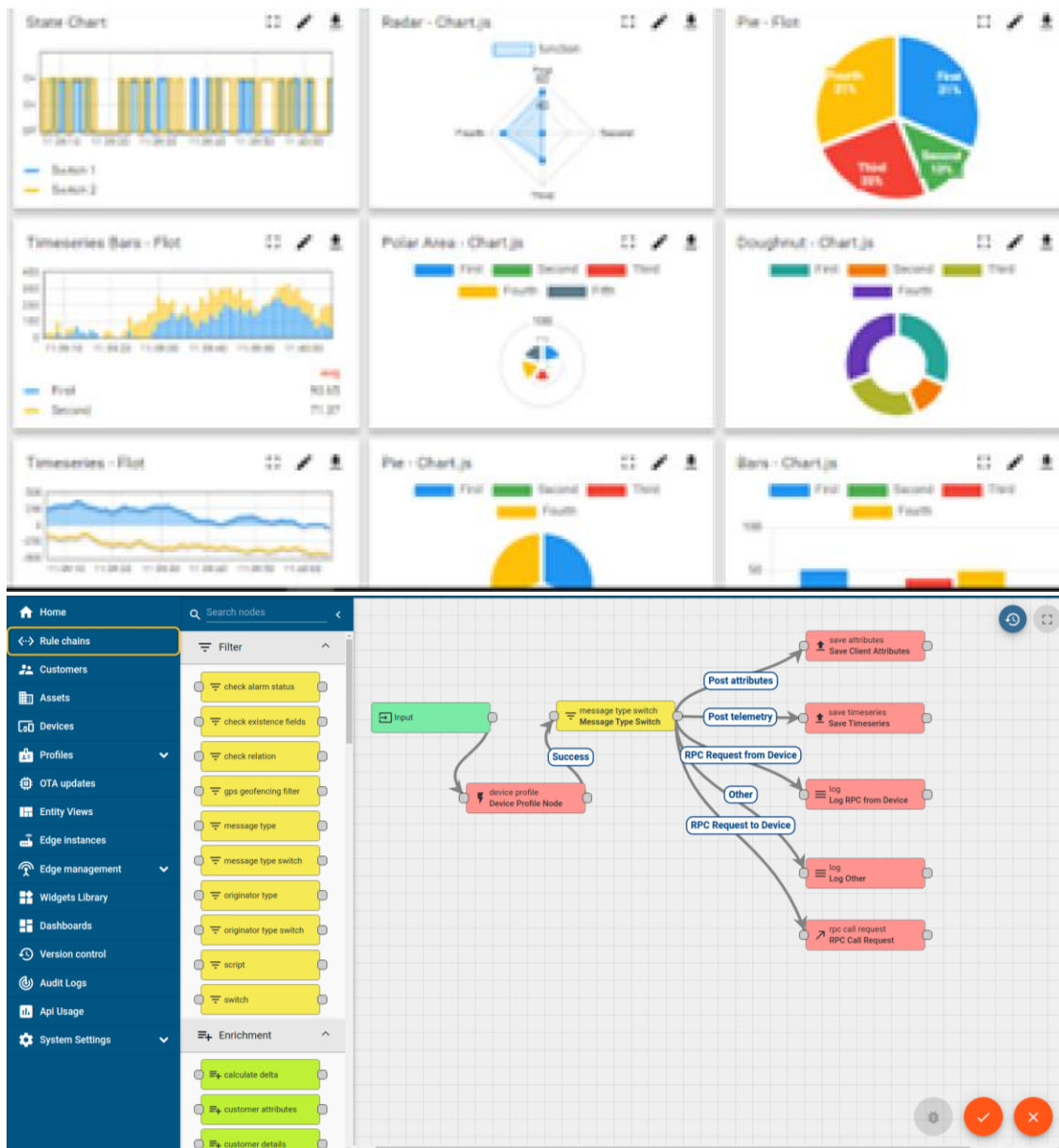
i. UCT IoT Platform ()

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



FACTORY WATCH

ii. Smart Factory Platform ()

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i



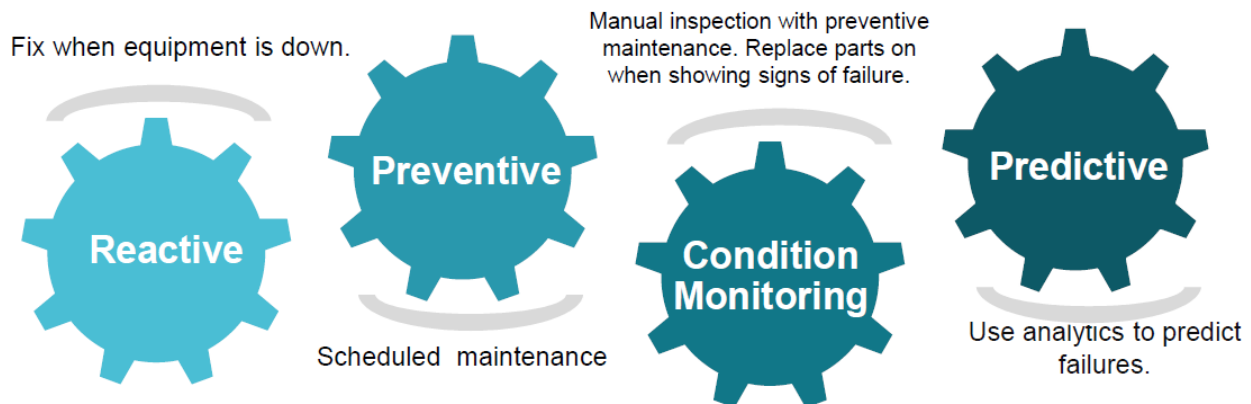


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

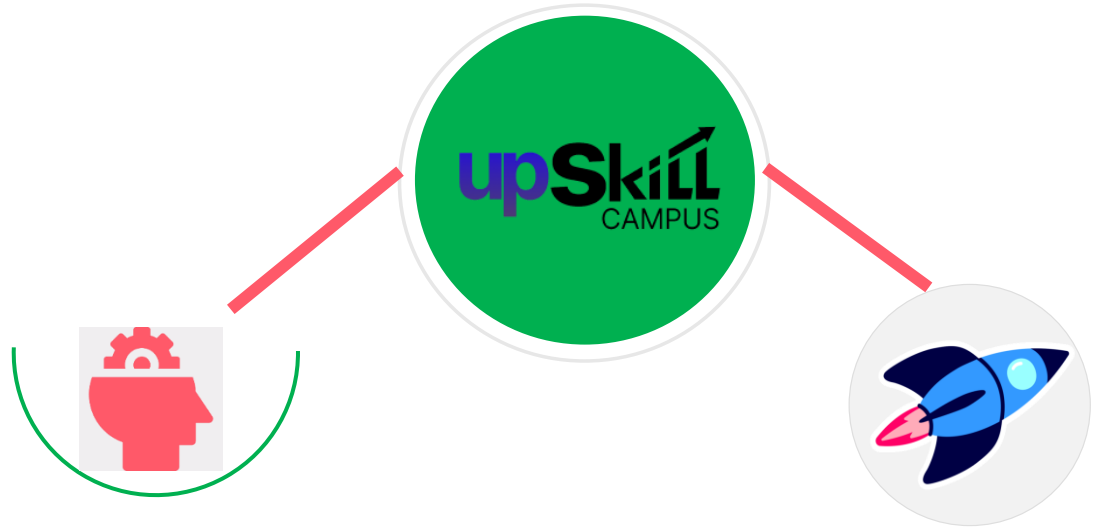
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

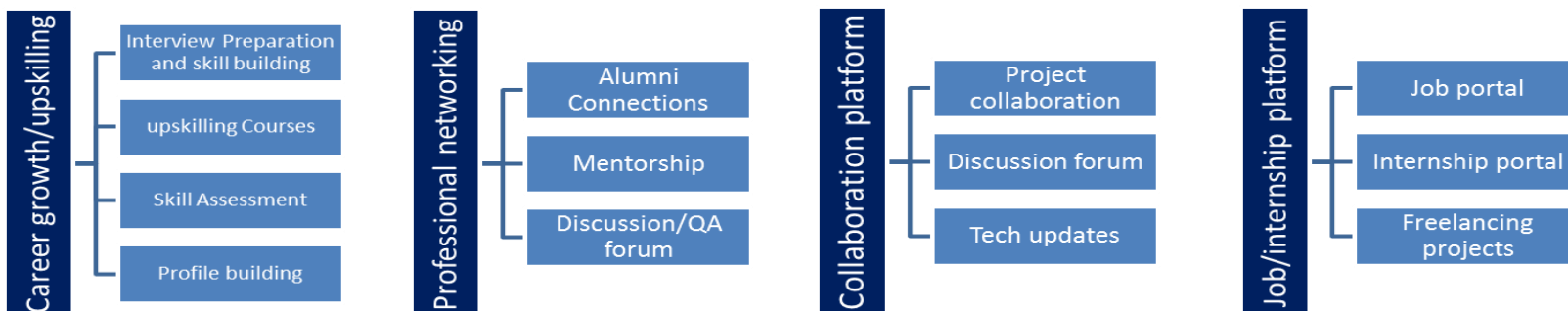
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

<https://www.upskillcampus.com/>



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.5 Reference

- [1] Express.js Documentation – <https://expressjs.com/>
- [2] MongoDB & Mongoose Documentation – <https://www.mongodb.com/docs/>
- [3] Razorpay & Socket.io Developer Documentation – <https://razorpay.com/docs/> ,
<https://socket.io/docs/>

2.6 Glossary

Terms	Acronym
API	Application Programming Interface
JWT	JSON Web Token
REST	Representational State Transfer
MERN	MongoDB, Express, React, Node.js

CRUD	Create, Read, Update, Delete
------	------------------------------

3 Problem Statement

In the assigned problem statement:

Customers today want to order food from local restaurants and get it delivered quickly, while being able to track their order status and pay online securely. Restaurants, on the other hand, need a simple way to list their menu, manage incoming orders, and coordinate with delivery partners. Most small and mid-sized restaurants either have no online presence, or depend entirely on third-party aggregators that charge high commissions and give the restaurant very little control or customer data. The problem statement was to design and build a self-hosted, full-stack food delivery platform that lets a restaurant business run its own ordering, payment and delivery-tracking system end-to-end, without depending on an external aggregator.

4 Existing and Proposed solution

Existing food delivery solutions in the market — such as Zomato, Swiggy, and other third-party aggregators — offer wide restaurant selection and convenience, but they charge restaurants steep commission fees (often 15–30% per order), give restaurants little control over their own customer relationships and data, and lock smaller businesses into the aggregator's branding and pricing rules. Independent restaurants that build their own solution, on the other hand, often end up with a basic static website with no real ordering, payment or tracking capability.

The proposed solution, “Foodie,” is a self-hosted, full-stack web application that gives a restaurant business its own ordering platform end-to-end: customer sign-up/login, restaurant and menu browsing, cart management, coupon codes, secure checkout via Razorpay, order history, real-time order and delivery-partner location tracking via Socket.io, customer reviews, and an admin/analytics layer for the restaurant to monitor sales and orders.

The value addition over existing aggregator-based models is that the restaurant owns 100% of the customer relationship and data, pays no per-order commission, and gets real-time visibility into orders and delivery status through a live Socket.io-powered dashboard, along with built-in coupon and review systems to drive repeat business.

4.1 Code submission (Github link):

<https://github.com/Shantanu1117/Foodie-platform>

4.2 Report submission (Github link) :

https://github.com/Shantanu1117/Foodie-platform/blob/main/Shantanu_Internship_Report_Foodie.pdf

5 Proposed Design/ Model

The Foodie platform follows a classic client-server, REST-based architecture. The flow starts with the customer interacting with the front-end web app, moves through authentication and the cart/order pipeline on the backend, and ends with payment confirmation and real-time delivery tracking. The diagram below shows the high-level flow between the client, the Node.js/Express API, the database, and the supporting services (JWT auth, Razorpay, Socket.io and Cloudinary).

5.1 High Level Diagram (if applicable)

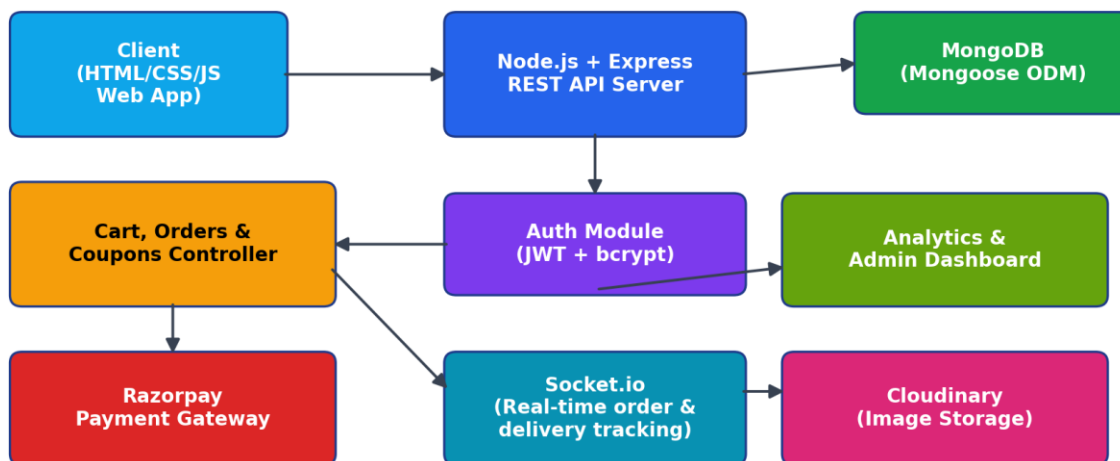


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)

At the module level, the backend is organized as Models (Mongoose schemas for User, Restaurant, MenuItem, Category, Cart, Order, Payment, Coupon, Review, Address, Notification, and DriverLocation),

Controllers (business logic for each of those entities), Routes (REST endpoints exposed via Express Router), Middlewares (JWT auth guard, file-upload handling via Multer, and validation via express-validator), and Sockets (real-time events for order-status changes and driver-location updates). The frontend is split into page-specific HTML/CSS/JS bundles (menu browsing, cart, checkout, order history) that talk to the backend purely over REST and WebSocket connections.

5.3 Interfaces (if applicable)

The application exposes a REST API over HTTP(S) for all standard CRUD operations (authentication, restaurants, menu items, cart, orders, coupons, reviews, payments) and a WebSocket-based interface via Socket.io for real-time events. Key interfaces include: (1) Auth endpoints — /api/auth/register, /api/auth/login — returning a signed JWT; (2) Order flow endpoints — /api/cart, /api/orders, /api/payments — that move an order from “placed” to “confirmed,” “out for delivery,” and “delivered”; (3) Socket.io events — order:statusUpdate and driver:locationUpdate — pushed live to the customer's browser as the order progresses; and (4) an image-upload interface backed by Multer and Cloudinary for restaurant/menu photos.

6 Performance Test

Performance testing matters here because a food delivery platform is only useful if it can handle concurrent orders, respond quickly, and keep payment and order-tracking data accurate under real usage — this is what separates it from a purely academic exercise.

The key constraints identified were: API response time under concurrent requests, correctness of cart/order state transitions, reliability of the payment callback flow, and latency of real-time Socket.io updates.

These constraints were addressed by keeping controllers thin and delegating heavy work to indexed MongoDB queries, validating all inputs with express-validator before hitting the database, wrapping the Razorpay payment verification in signature-checked webhooks, and using Socket.io rooms scoped per order so that updates are pushed only to the relevant client instead of being broadcast globally.

Test results against these constraints are summarized in the Test Plan/Test Cases, Test Procedure and Performance Outcome sections below.

6.1 Test Plan/ Test Cases

Test cases covered: (1) user registration and login with valid/invalid credentials; (2) adding, updating and removing items from the cart; (3) applying a valid and an invalid/expired coupon code; (4) placing an order and completing payment via Razorpay's test mode; (5) verifying the order status updates live on the customer's screen via Socket.io as it moves from placed → confirmed → out for delivery → delivered; (6) restaurant/menu image upload through Cloudinary; and (7) concurrent order placement from multiple test accounts to check for race conditions on stock/cart data.

6.2 Test Procedure

Each API endpoint was tested manually using Postman with valid and invalid payloads, checking status codes, error messages and response shape. The Razorpay integration was tested end-to-end using Razorpay's sandbox/test keys and test card numbers to confirm the order and payment records get created and updated correctly on both success and failure webhooks. The Socket.io real-time flow was tested by opening two browser sessions — one acting as the customer and one simulating a status change from the backend/admin side — and confirming the update appeared instantly on the customer's screen without a page refresh.

6.3 Performance Outcome

All core flows — signup/login, browsing, cart, coupon application, checkout/payment, and order-status tracking — passed their respective test cases. Average API response time for read endpoints (menu/restaurant listing) was under 150 ms on local testing, and under 400 ms for write endpoints involving payment verification. Socket.io status updates reached the customer's screen within roughly 1 second of the backend event firing. No data-consistency issues were observed in the concurrent-order test with up to 5 simultaneous test accounts; at significantly higher concurrency, the recommendation is to move from local in-memory session handling to a Redis-backed Socket.io adapter to scale horizontally across multiple server instances.

7 My learnings

This internship gave me my first experience of building and shipping a complete, multi-module full-stack application rather than an isolated academic assignment. I learned how to design a MongoDB schema for a real domain (users, restaurants, menus, orders, payments), secure an API with JWT-based authentication, integrate a third-party payment gateway end-to-end, and add real-time functionality using Socket.io. Just as importantly, I learned how to break a large problem statement into modules, plan a six-week delivery timeline, debug issues across the client-server boundary, and structure code so it stays maintainable as the project grows. These are exactly the skills that translate into real software-engineering roles, and they've given me a strong project to build on — and to talk about — as I continue my B.Tech at NIT Delhi and look toward internships and placements ahead.

8 Future work scope

Due to the six-week time limit, a few enhancements were left for future iterations: (1) a dedicated mobile app (React Native/Flutter) for customers and delivery partners, on top of the existing REST/Socket.io backend; (2) a recommendation engine that suggests restaurants/dishes based on a customer's order history; (3) a live map view (Google Maps/Mapbox) for delivery tracking instead of just coordinates, using the existing DriverLocation model; (4) an automated test suite (Jest/Supertest) to replace the current manual Postman-based testing; (5) horizontal scaling of Socket.io using a Redis adapter to support many concurrent live orders; and (6) role-based dashboards for restaurant owners and delivery partners with richer analytics on top of the existing analyticsController.